

Lógica y Representación 1

Algoritmos y diagramas de flujo

Ingeniería de Sistemas

Universidad de Antioquia

Resolución de problemas

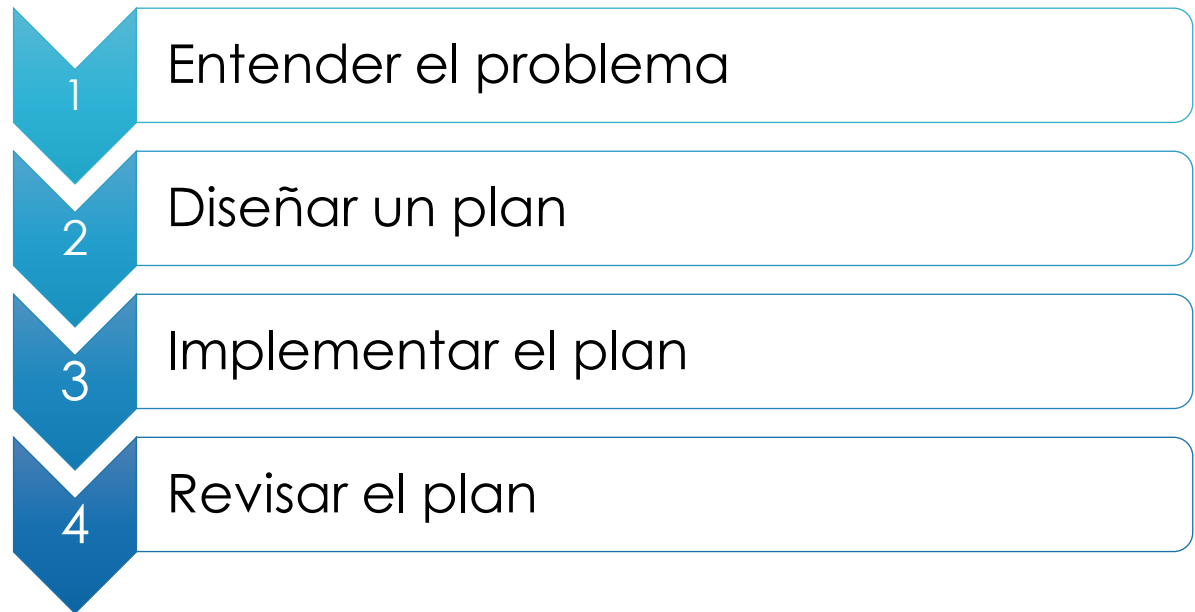
A partir del planteamiento de un problema, es conveniente usar una **metodología** para la resolución del problema, que tendrá como objetivo final el algoritmo que dará una solución.



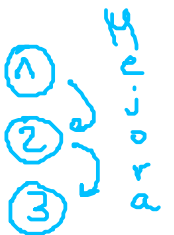
Homero + cerveza = Presidente ([link](#))

Método de Polya

George Polya, Matemático húngaro, autor del libro ***How to solve it*** (1945). En él, Polya propone cuatro pasos generales para la solución de un problema



Ejemplo: Algoritmo para ir a comprar un
tinto en una burbuja de la UdeA



Método de Polya

1. Entienda el problema: parece obvio pero es con frecuencia un gran obstáculo.

- ¿Entiende todas las palabras de la formulación del problema?
- ¿Qué le están pidiendo que encuentre?
- ¿Puede usted reformular el problema en sus propias palabras?
- ¿Puede hacer un dibujo que represente el problema?
- ¿Hay suficiente información para encontrar la solución?

Método de Polya

2. **Diseñe un plan:** escoja una estrategia para resolver el problema (divida el problema en problemas más simples, elimine posibilidades, aproveche simetrías, suponga y verifique, etc).
3. **Implemente el plan:** más fácil que el paso 2, sólo requiere mucho cuidado en los detalles y paciencia.
4. **Revise:** haga una pausa, revise y reflexione sobre el trabajo hecho.

Algoritmos

Recordemos que, un algoritmo es un proceso preciso, computable y finito que paso a paso lleva a la solución de un problema.

Todo algoritmo debe tener tres partes:



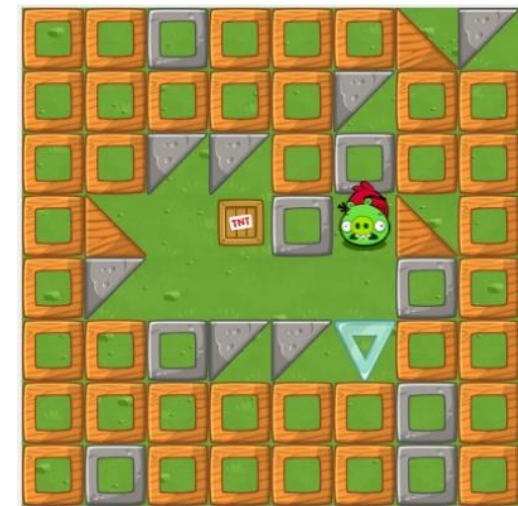
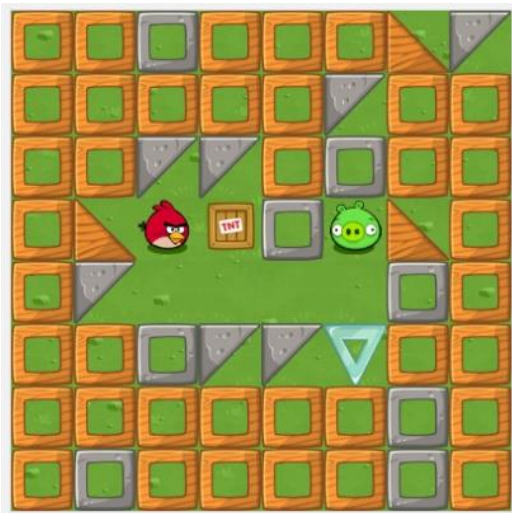
Algoritmo

Teniendo en cuenta los bloques que se muestran a la derecha, describa el algoritmo que permita que el pájaro pise al marrano ([link](#)).

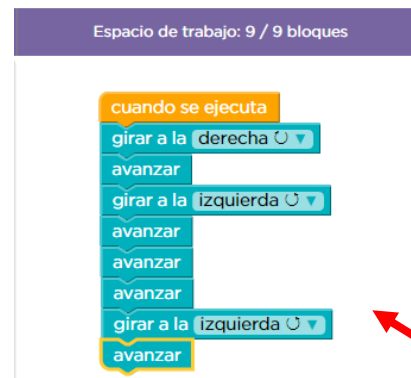


Algoritmo - Solución

Teniendo en cuenta los bloques que se muestran a la derecha, describa el algoritmo que permita que el pájaro pise al marrano ([link](#))



Bloques



Implementación
del Algoritmo

Método de Polya - Resumen



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

Revisar el plan

Datos en algoritmos

Variable: espacio de memoria asociado con un identificador (nombre) que almacena un valor que puede ser modificado por instrucciones del algoritmo:

- De entrada y salida.
- Variables auxiliares.

```
x = 3
y = 6
y = y + 2*x
x = x + 1
z = x - 2*(y - 1)
Z = -z
```

Ejemplo:

Dado el conjunto de instrucciones mostrado a en el cuadro:

- ¿Cuántas variables hay?
- ¿Cuáles son esas variables?
- ¿Cuál es el valor final de estas?

[link](#)

Datos en algoritmos

- Una **variable** puede representar un número decimal, un numero entero, un arreglo de números o de caracteres, etc.

Grupo	Tipo	Descripción	Ejemplo
Numéricas	Enteros	Variables asociadas a números enteros, es decir, sin parte decimal	5, 6, -12, 3000,...
	Reales	Variables asociadas a números reales (con parte decimal).	0.08, 10.0, -1.5, 300.26,...
Lógicas	Booleanas	Variables que pueden tomar dos posibles valores	True (Verdadero), False (Falso)
Alfanuméricas	Carácter	Variables que pueden tomar solo un conjunto finito de valores reconocidos por computador	tabla ascii , tabla Unicode , etc.
	Cadena (Texto)		

- Declaración de variables:** Consiste en asignarle un nombre (identificador) a un espacio en memoria donde se va a guardar un valor

Datos en algoritmos

- **Declaración de variables:** Consiste en asignarle un nombre (identificador) a un espacio en memoria donde se va a guardar un valor

Entero, Real, Texto, Logico

↓

Valor inicial

↓

<Tipo_de_dato> identificador [= valor]

↑

Nombre de la variable

```
Entero edad = 25
Real peso = 59.5
Texto nombre = "Pepe"
```

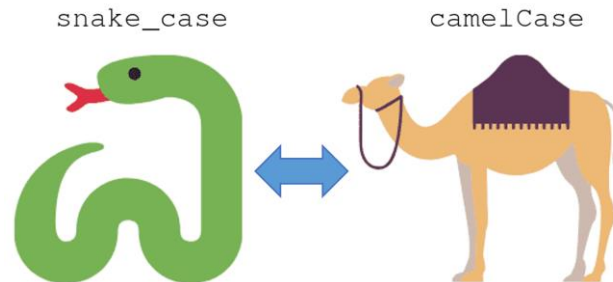
Memoria	
edad	25
peso	59.5
nombre	"pepe"
...	

Datos en algoritmos

Los **identificadores** son nombres que se dan a los elementos utilizados para resolver un problema (variables, funciones, procedimientos, nombre del programa).

Reglas y buenas practicas	Ejemplos
Pueden estar formados por una combinación de números y letras. Sin embargo, el primer carácter no puede ser un numero.	<code>grupo1</code> , <code>grupo_2</code> , <code>ced</code> , <code>peso</code> , <code>num_est</code> , <code>nom</code> , <code>name</code> , <code>_x</code> , <code>y</code> , <code>2345bcc</code> , <code>num_est</code> , <code>€+23</code>
Usar nombres autodescriptivos, los nombres de las variables deben indicar claramente qué almacenan.	Nombre Completo → <code>nombre</code> Numero de estudiantes → <code>numero_estudiantes</code>

Estilos de nombrado para juntar múltiples palabras en una sola:



Buenas prácticas

- Las buenas prácticas ayudan a escribir código más legible, mantenible y eficiente. Además de facilitar a otros desarrolladores la lectura y seguimiento de este
- Cada lenguaje tiene sus propias convenciones ([PEP 8 - Style Guide for Python Code](#)).

Regla	Ejemplo correcto	Ejemplo a evitar
Nombres descriptivos, no abreviados en exceso	numero_estudiantes	n
Sin tildes ni ñ (evita errores de codificación)	calcular_promedio	calcularPromédio
Sin mezclar estilos	total_ventas	Total_Ventas
Verbos para funciones, sustantivos para variables	<code>def validar_correo():</code>	<code>def correo_validado_funcion():</code>
Evitar nombres reservados de Python	lista_datos	list, dict, for

Datos en algoritmos

Ejemplo:

Dada la siguiente tabla de Datos. Que nombres marque con una X la columna A (apropiados) o NA (No apropiados) si los nombres de las variables son los apropiados para estos datos. En caso de que no sean los apropiados, sugiera un nombre.

#	Dato	Nombre Variable	A	NA	Nombre sugerido
1	Temperatura	jj			
2	Peso	p			
3	Estatura	heigth			
4	Edad	vejez			
5	Numero de gallinas	ng			
6	Horas	h			
7	Cantidad de carros	car			
8	Nombre	n			

Datos en algoritmos

Ejemplo:

Dadas las siguientes variables, defina el tipo de dato al que pertenecen marcando con una X la columna mas apropiada

Dato	Nombre Variable	Entero	Real	Booleana	Carácter	Cadena
Dinero	money					
Correo electrónico	email					
velocidad	speed					
Numero de llamadas	num_calls					
Sexo	gender					
Radio	radio					
Numero de materias	num_materias					
Indice de masa corporal	imc					

Datos en algoritmos

- **Constante:** Dato del algoritmo que luego de ser asignado, que no puede ser modificado
- Para nombrar constantes se usan mayúsculas

Dato	Nombre de la constante
Numero Pi	PI = 3.14
Numero Euler	exp = 2.71828
Gravedad	G = 9.8
Velocidad de la luz	C = 300000
Numero de Avogadro	NUM_AVOGADRO = 6.022e-23
Limite de intentos	LIMITE_INTENTOS = 30

- **Declaración:**

Entero, Real, Texto, Logico

Valor de la constante

<Tipo_de_dato> identificador = valor
Nombre de la variable

```
Entero LIMITE_INTENTOS = 30  
Real PI = 3.14
```

Diagrama de flujos - Bloques

Inicio
10/08/2026

Inicio del programa

Start



Entrada/Salida

Read/Print



Pasos, procesos o líneas
de instrucción de
programa de computo

Process



Luego: 1. Condiciones
2. Ciclos

Toma de decisiones y Ramificación



Líneas de flujo



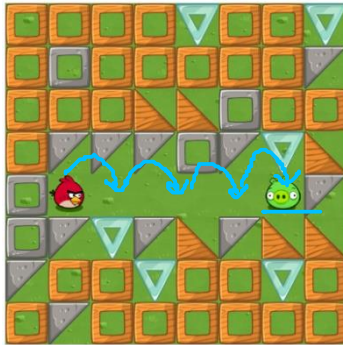
Fin del programa

Finish



Comparación – Solución del problema del pájaro

Problema: Pise al marrano con el pájaro



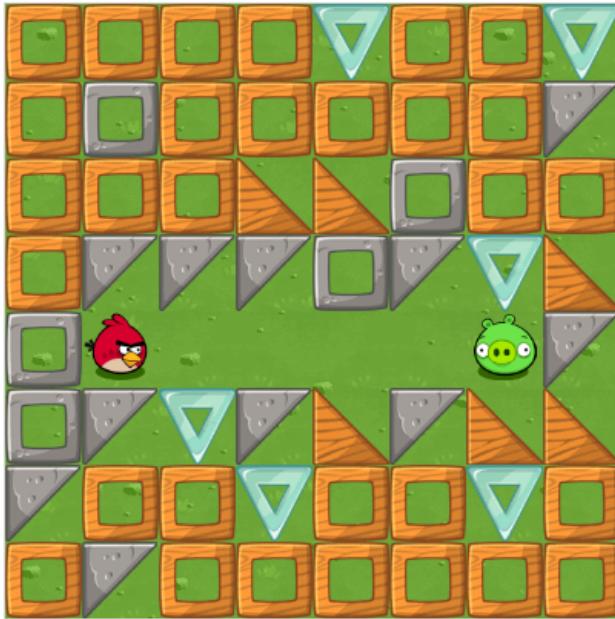
Bloques problema del pájaro



Algoritmo Solución

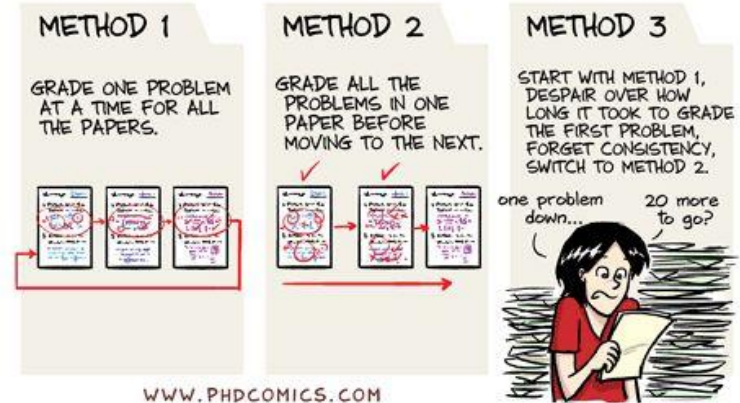


Comparación de un problema



GRADING
METHODS

JORGE CHAM © 2010



Problema:

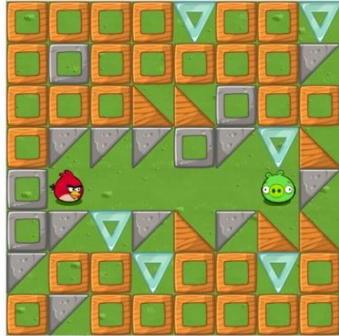
Haga un programa que ...

Problema:

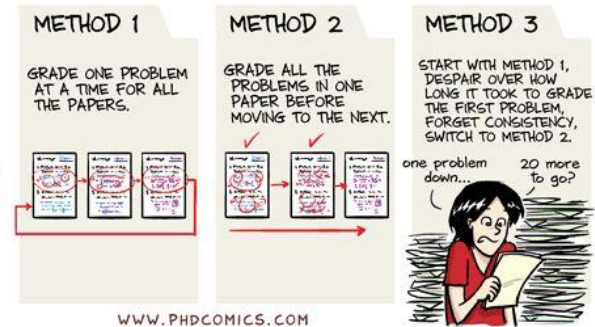
Pise al marrano con el pájaro

Como
escribo
algoritmos?

Comparación de un problema



GRADING METHODS



JORGE CHAM © 2010

WWW.PHDCOMICS.COM

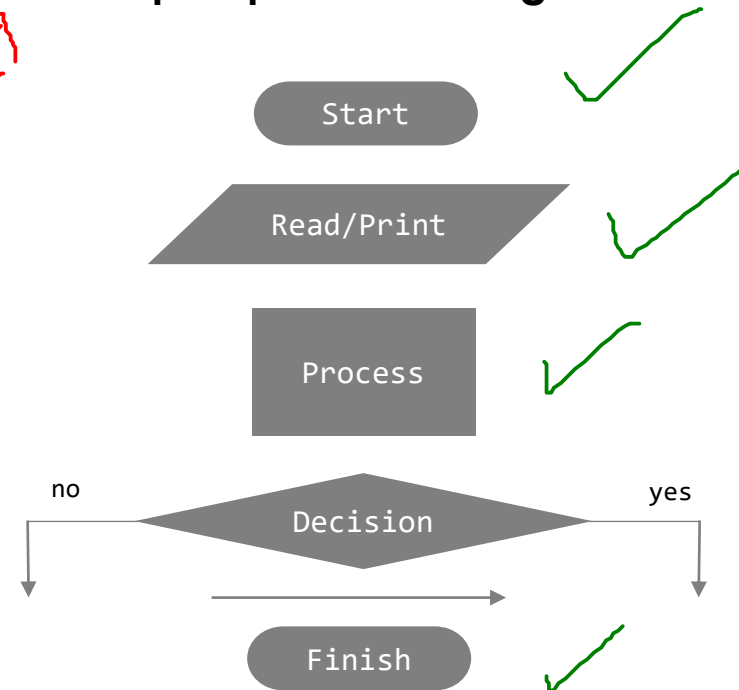
Bloques problema del pájaro



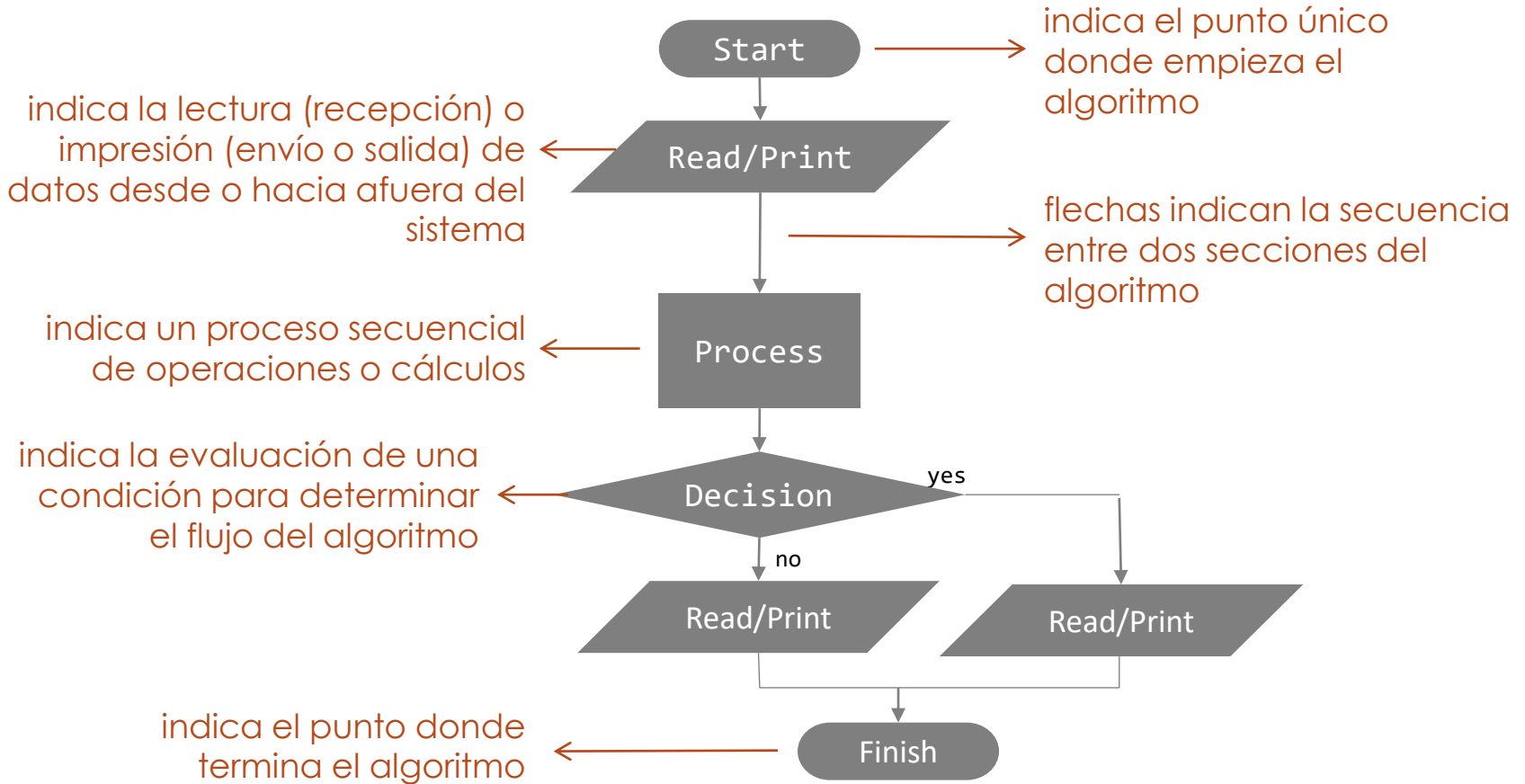
Bloques problema asignado



Lue go



Diagramas de flujo (Flow charts)



Bloques básicos

Bloques vistos hasta el momento:

- **Entrada/Salida:**



Read/Print

- **Proceso:**



Process

- **Condicionales:**



Decision

no

yes

Programacion
estructurada

Que hace
cada
bloque?

Entrada

Entrada/Salida

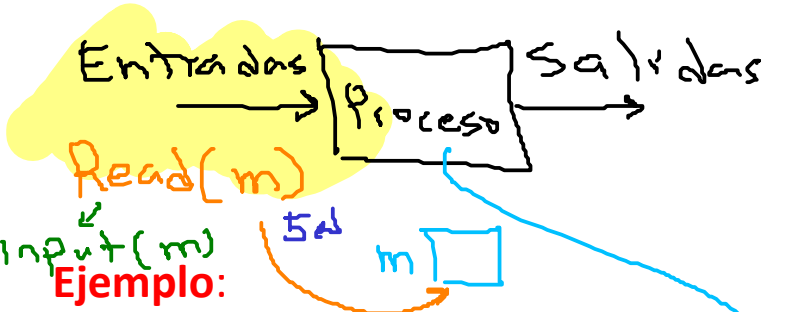
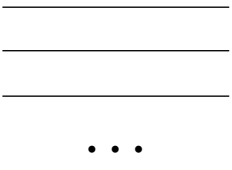
Read/Print



Read



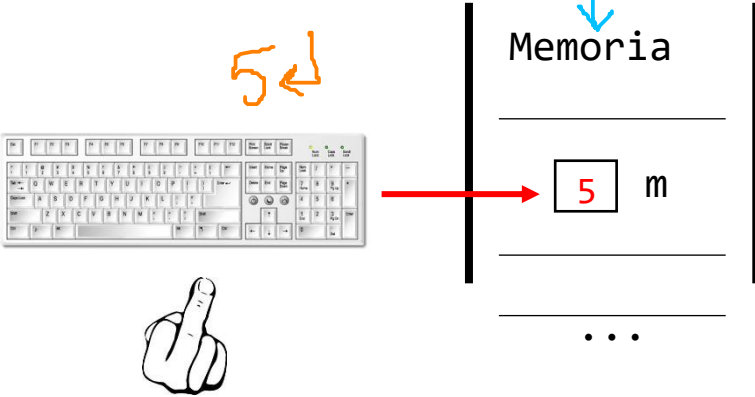
Memoria



Dado el siguiente bloque cual seria su efecto:

Read m

Solución:



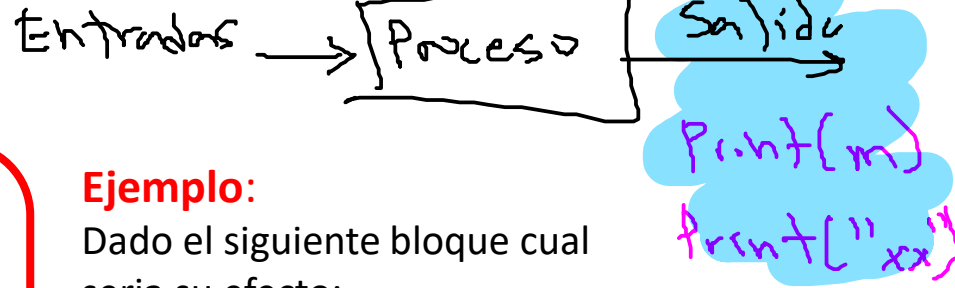
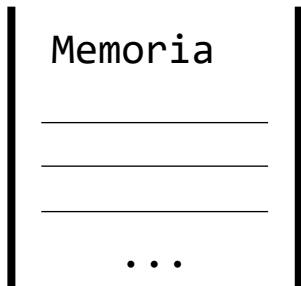
Salida

Entrada/Salida

Read/Print



Print

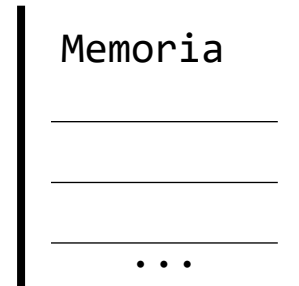


Ejemplo:

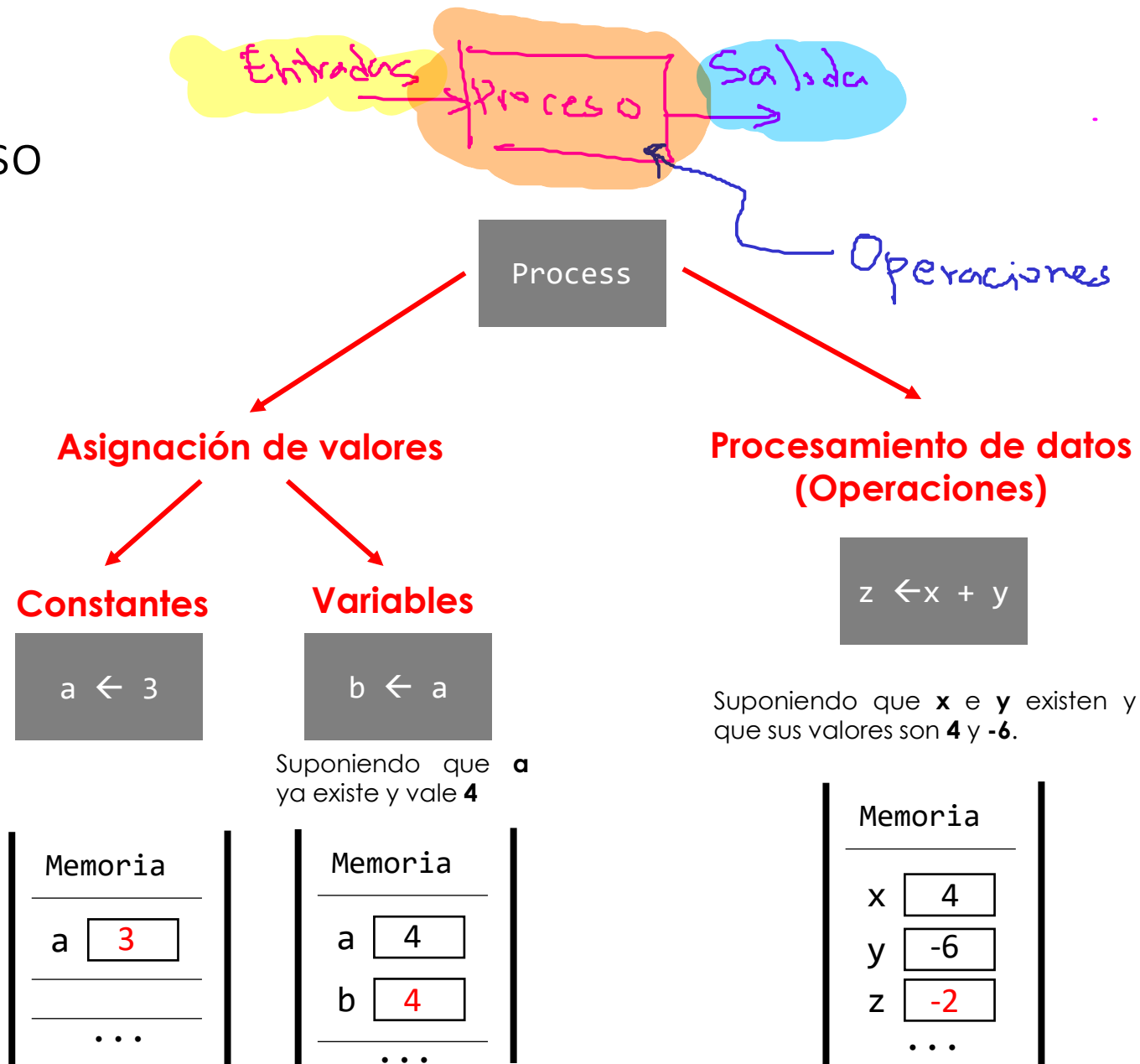
Dado el siguiente bloque cual seria su efecto:

Print "Hola"

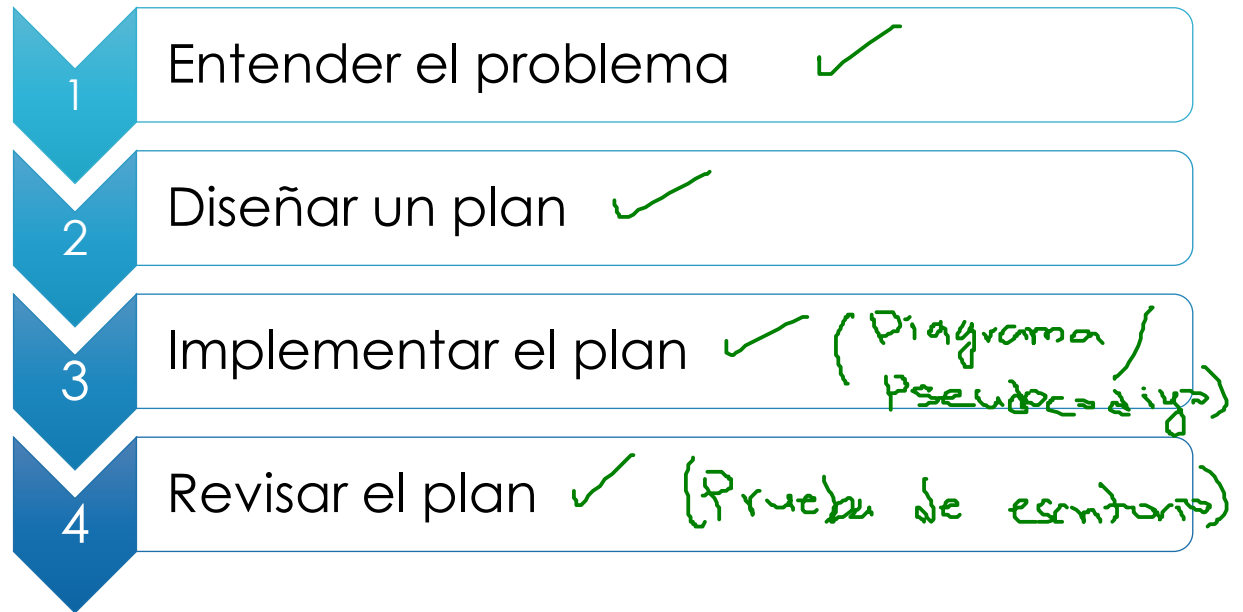
Solución:



Proceso



Método de Polya - Resumen



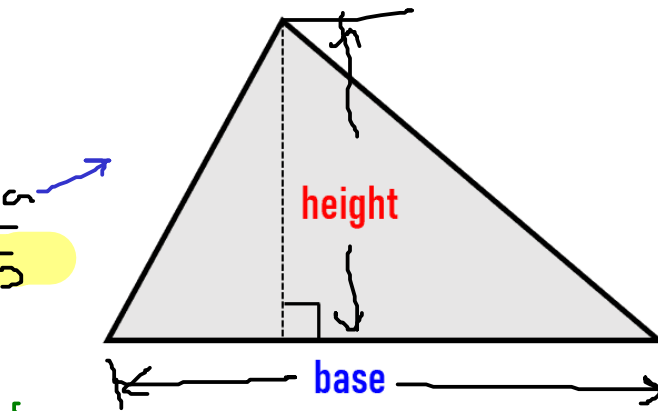
Ahora que tenemos una forma para representar un algoritmo, procedamos a aplicar el método de Polya al diseño de algoritmos.

Diseño de Algoritmos

Ejemplo 1: Realice un programa que permita calcular al área de un triángulo

$$\text{area} = \frac{\text{base} \times \text{height}}{2}$$

base	height	area
5	5	12.5
6	8	24



Como
hago el
algoritmo?

Entradas

base
height

Proceso

Salidas

area

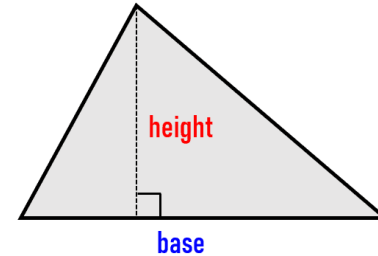
$$\text{area} = (\text{base} * \text{height}) / 2$$

Método de Polya - Resumen



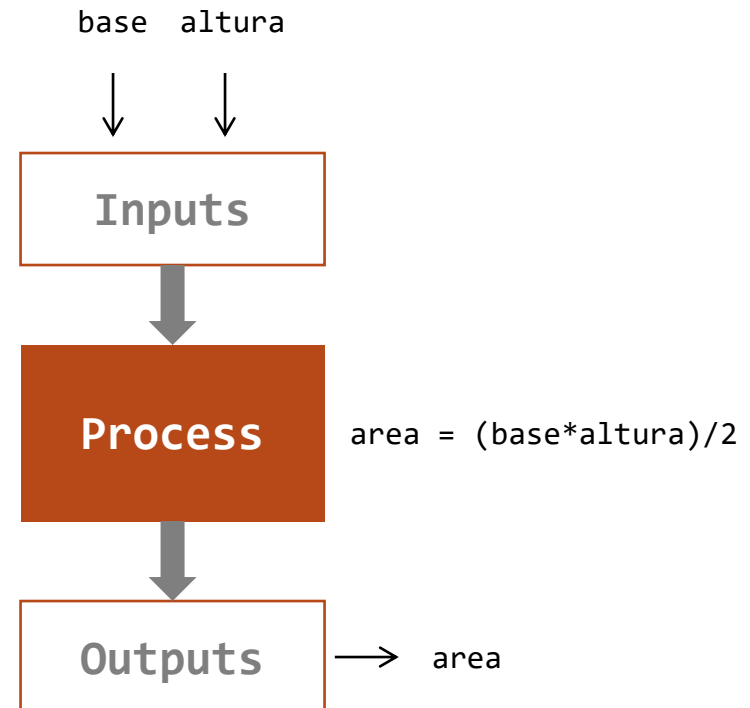
1

Entender el problema

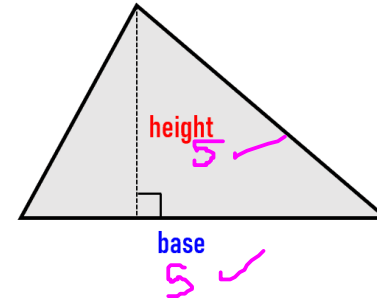
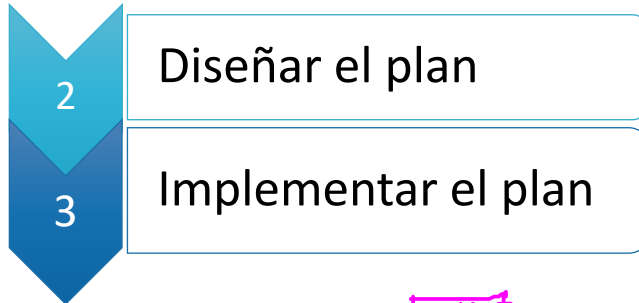


Análisis:

- ¿Cuál es el objetivo buscado?
✓ Calcular el área de un triángulo
- ¿Cuáles son los datos de entrada?
✓ Base y altura
- ¿Cuáles son los datos de salida?
✓ Área de un triángulo
- ¿Qué cálculos/procesos deben llevarse a cabo?
✓ Multiplicar la base por la altura y dividir entre 2



Representación de algoritmos como diagramas de flujo



- **Datos de entrada:** longitud de los lados

- **Datos de salida:** área del rectángulo

- **Definición de variables:**

- **base:** base (Real)
- **height:** alto (Real)
- **area:** área (Real)

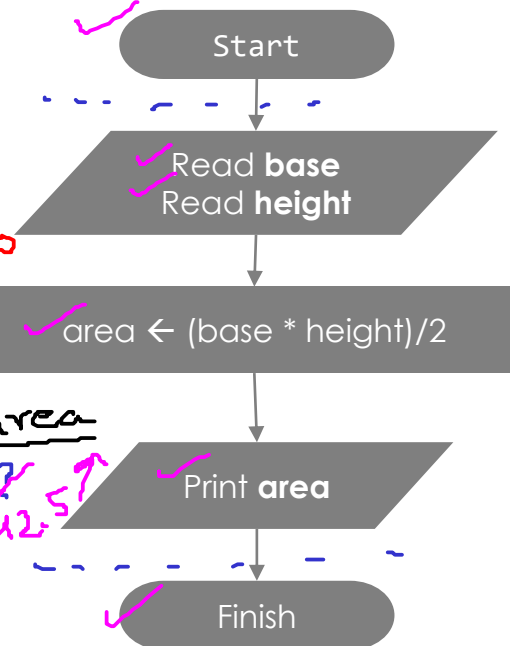
- **Proceso:**

- **$area = (base * altura) / 2$**

↓ =  *teclado*

Prueba de escritorio

base	height	area
5	5	12.5



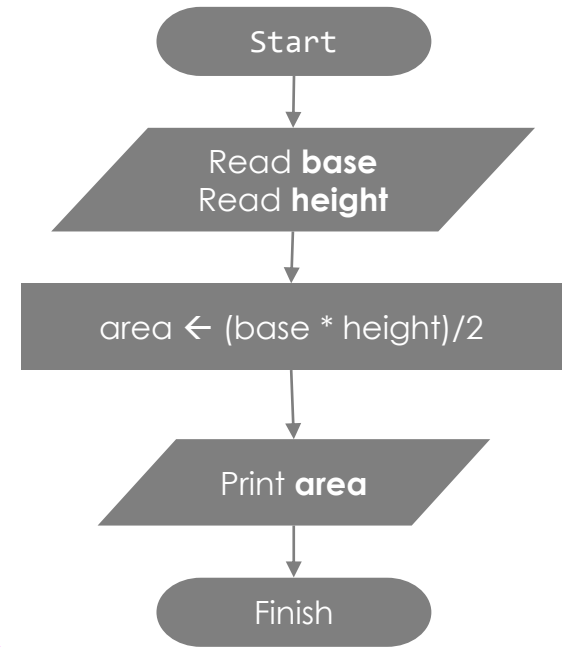
↗ =  *Pantalla*

output

12.5

Prueba de escritorio

- Una **prueba de escritorio** (trace table) es una técnica manual para simular la ejecución de un algoritmo, sin usar el computador.
- Es importante porque:
 - Permite verificar la lógica del algoritmo antes de traducirlo a código.
 - Ayuda a detectar errores lógicos antes de programar.
 - Refuerza la comprensión del flujo de ejecución.



Prueba:

Entradas		Salidas (esperadas)
base	Height	area
4	3	6

↘base	↘height	↗area
?	?	?
↘4	↘3	↗6

Representación de algoritmos como diagramas de flujo



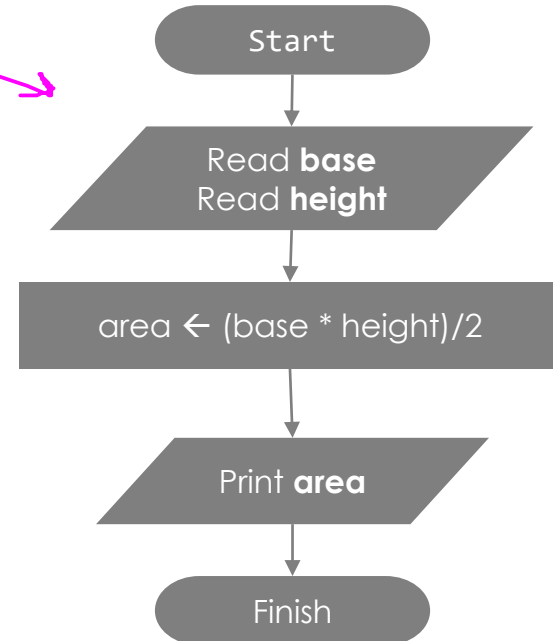
4

Revisar el plan

Prueba:

Entradas		Salidas (esperadas)
base	Height	area
3	5	7.5

∇ base	∇ height	$\nearrow$ area
$\nabla 3$	$\nabla 5$	$\nearrow 7.5$



Diseño de Algoritmos

Ejemplo 2: Se requiere diseñar un algoritmo que calcule el número de meses que hay entre los años A y B.

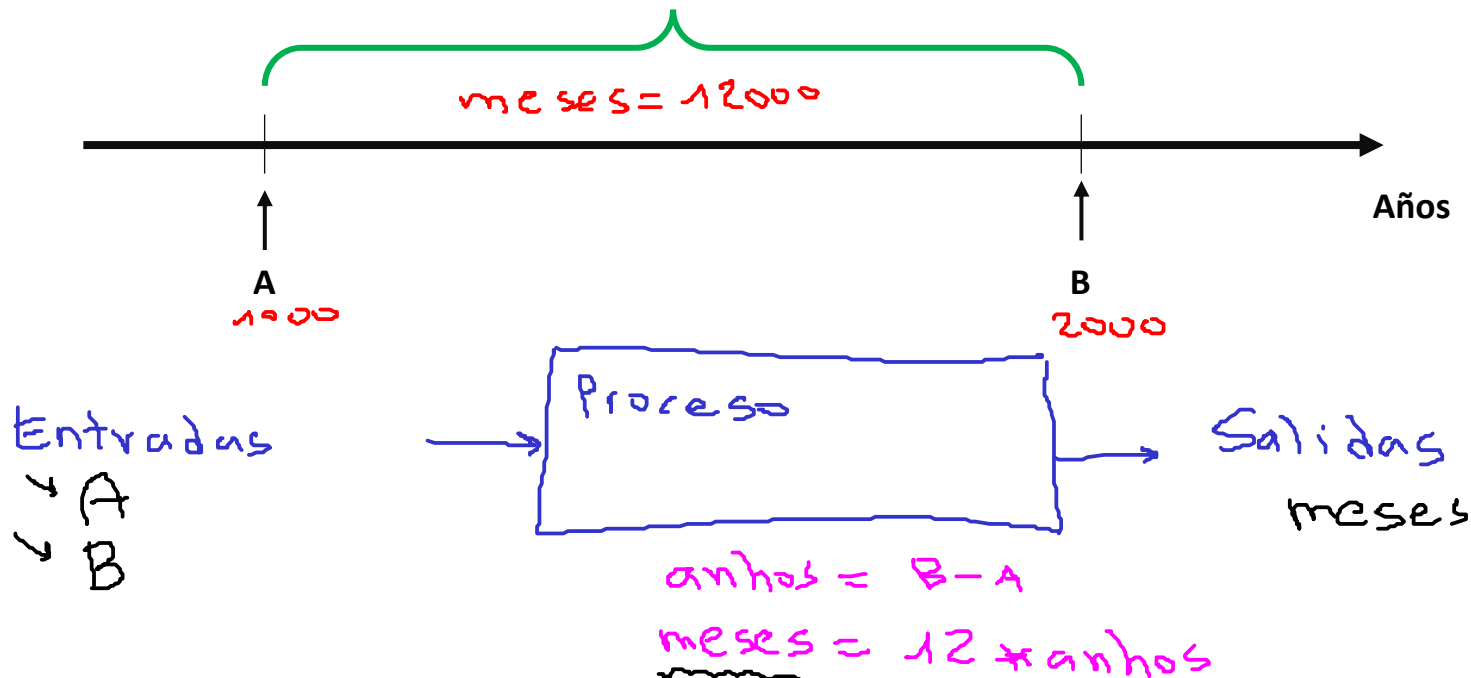
Formula:

$$\text{años} = (B - A)$$
$$\text{meses} = 12 * \text{años}$$

Handwritten calculation example:

1000 (A) 2000 (B)

$$\text{years} = \frac{2000 - 1000}{1} = 1000$$
$$\text{months} = 12 \times 1000 = 12000$$



Diseño de Algoritmos

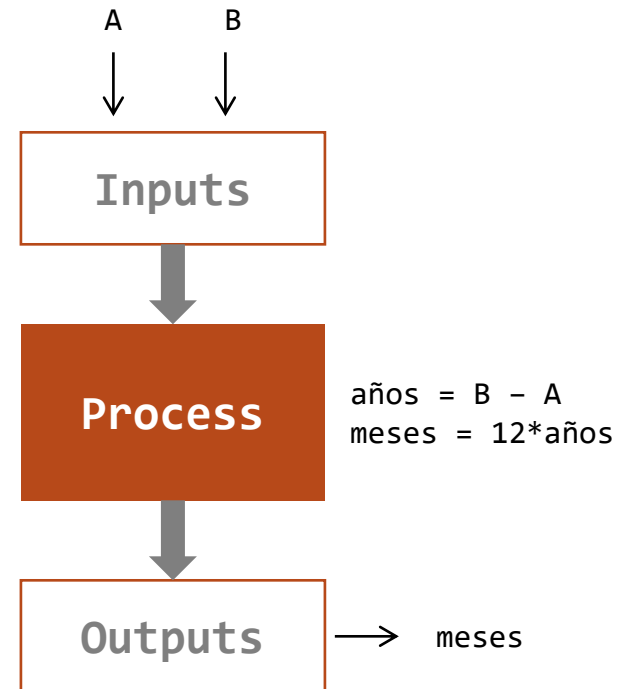
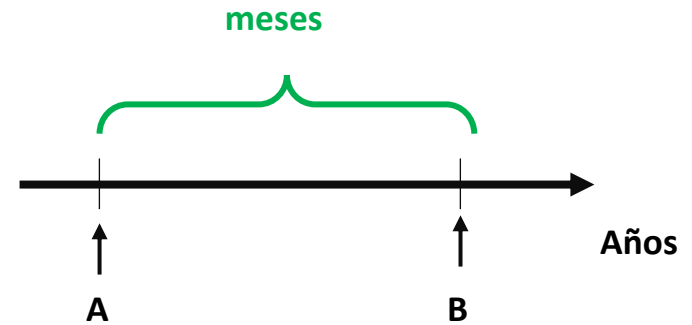


1

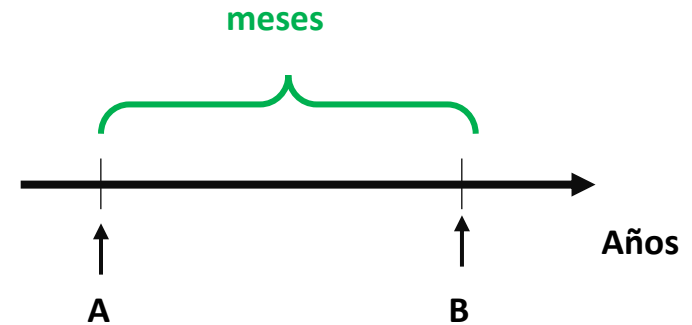
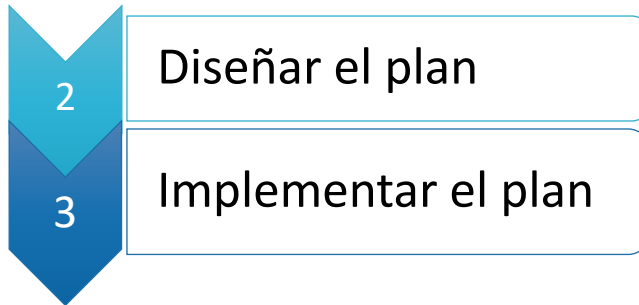
Entender el problema

Análisis:

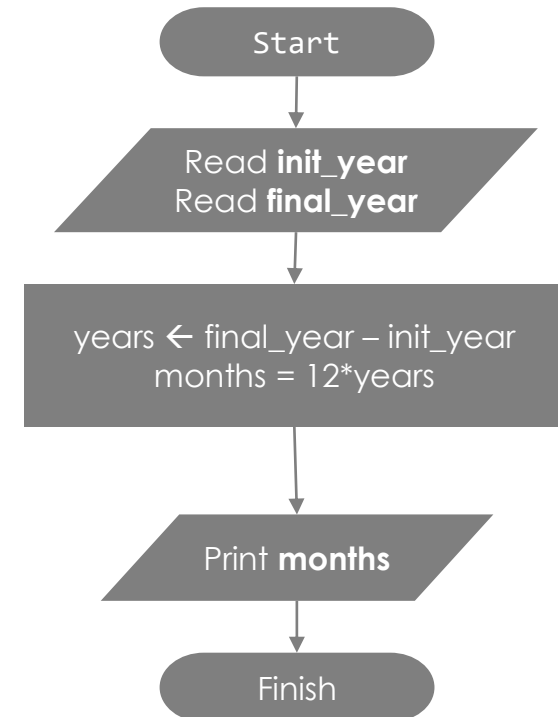
- ¿Cuál es el objetivo buscado?
Calcular los meses entre dos años.
- ¿Cuáles son los datos de entrada?
Año inicial (A) y año final (B)
- ¿Cuáles son los datos de salida?
Cantidad de meses
- ¿Qué cálculos/procesos deben llevarse a cabo?
 $\text{años} = B - A$
 $\text{meses} = 12 * \text{años}$



Representación de algoritmos como diagramas de flujo



- **Datos de entrada:** los años especificados (A y B)
- **Datos de salida:** numero total de meses transcurridos
- **Definición de variables:**
 - ***init_year***: año inicial (A) (Entero)
 - ***final_year***: año final (B) (Entero)
 - ***years***: años transcurridos (Entero)
 - ***months***: meses transcurridos (Entero)
- **Proceso:**
 - $years = final_year - init_year$
 - $months = 12 * years$



Representación de algoritmos como diagramas de flujo



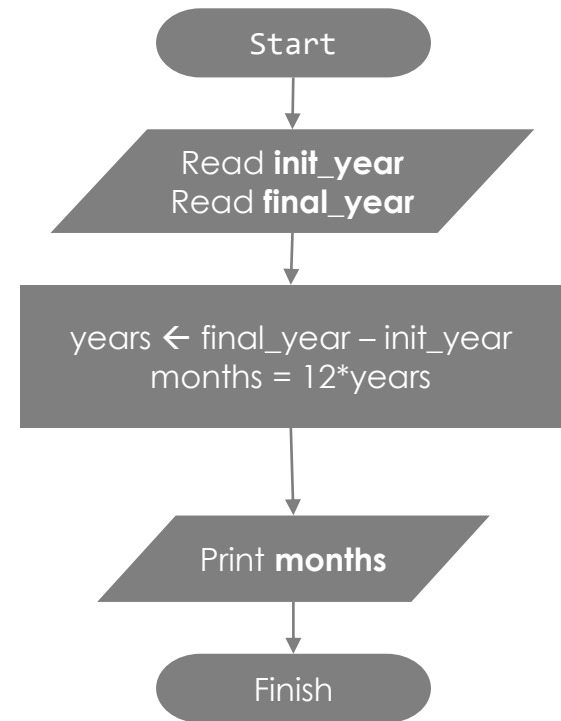
4

Revisar el plan

Prueba:

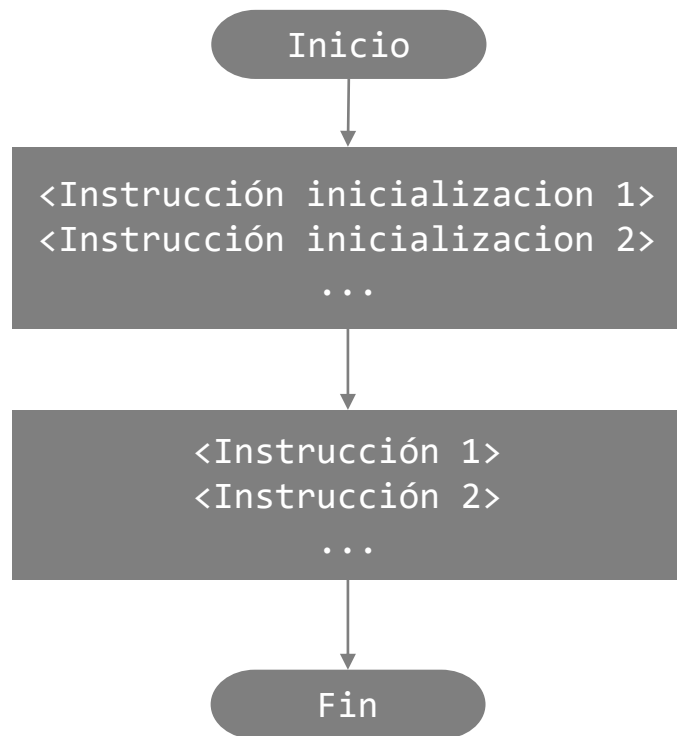
Entradas		Salidas (esperadas)
init_year	final_year	months
4	10	72

∇ init_year	∇ final_year	years	$\nearrow$ months
∇ 4	∇ 10	7 6	? $\nearrow$ 72



Pseudocódigo

- El **pseudocódigo** es una descripción informal de un algoritmo, escrita en lenguaje natural estructurado, que utiliza palabras clave similares a un lenguaje de programación pero sin una sintaxis estricta. (Notación del profesor Efraín Oviedo).
- Sigue una estructura básica de **inicio**, **inicialización**, **cuerpo** y **fin**.



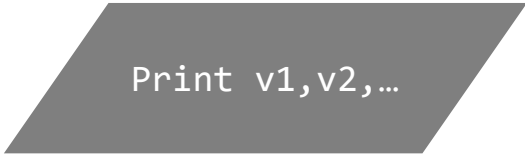
INICIO

```
# Inicialización de variables
<Instrucción inicialización 1>
<Instrucción inicialización 2>
...
# Cuerpo del algoritmo
<Instrucción 1>
<Instrucción 2>
...
```

FIN

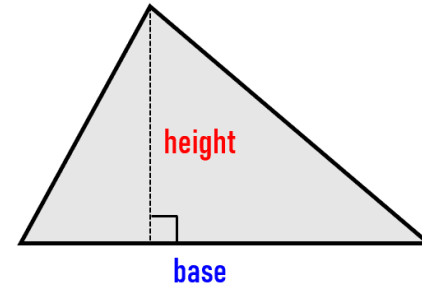
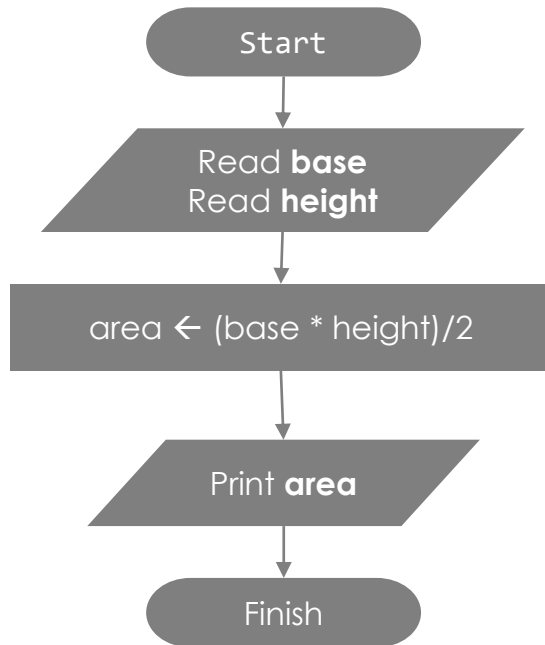
Pseudocodigo

La siguiente tabla muestra el resultado equivalente para operaciones secuenciales básicas entre el diagrama de bloques y el pseudocódigo (con las estructuras vistas hasta el momento).

Tipo	Diagrama de bloques	Pseudocordigo
Entrada 		Leer(v1,v2,...)
Salida 		Escribir(v1,v2,...)
Proceso 		$c = a + b$

Representación de algoritmos mediante Pseudocódigo

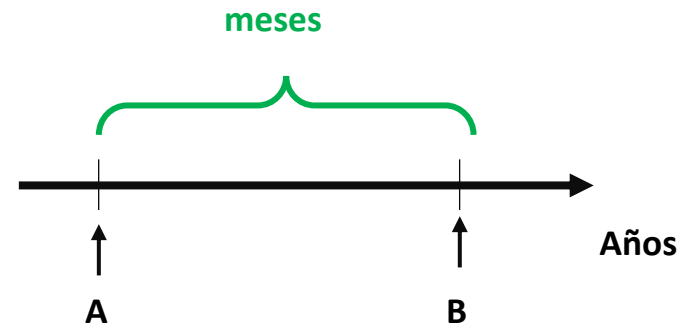
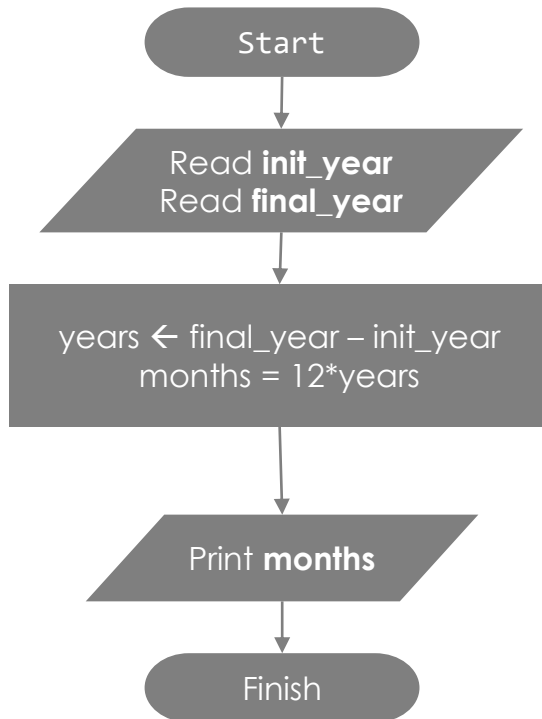
Ejemplo 1: Realice un programa que permita calcular al área de un triángulo



```
Inicio  
Leer(base)  
Leer(height)  
area = (base*height)/2  
Escribir(area)  
Fin
```

Representación de algoritmos mediante Pseudocódigo

Ejemplo 2: Realice un programa que permita calcular al área de un triángulo



Inicio

```
Leer(init_year)
Leer(final_year)
years = final_year - init_year
months = 12*years
Escribir(months)
```

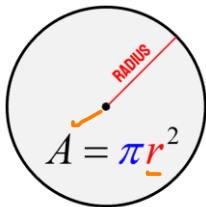
Fin

Resumiendo del proceso de diseño del algoritmos

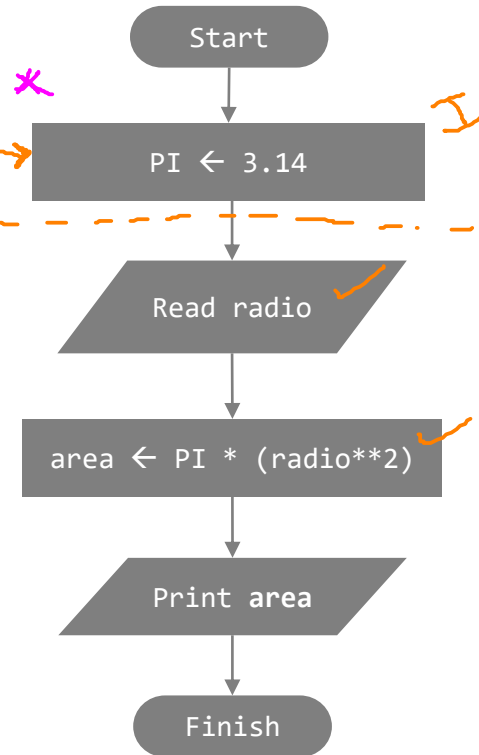
Ejemplo 3:

Hacer un algoritmo que calcule el área de un círculo

1



3



Inicio

PI = 3.14
Leer(radio)
area = PI*(radio**2)
Escribir(area)

Fin

4

Inicialización

Entradas		Salidas	
radio		area	
10		314	
2.6		21.237	

↘radio	↗area
?	?
↘10	↗314

↘radio	↗area
?	?
↘2.6	↗21.226

2

Análisis:

Datos de entrada: radio

Datos de salida: área del círculo

Constantes:

- **Numero PI:** PI = 3.14 (Real)

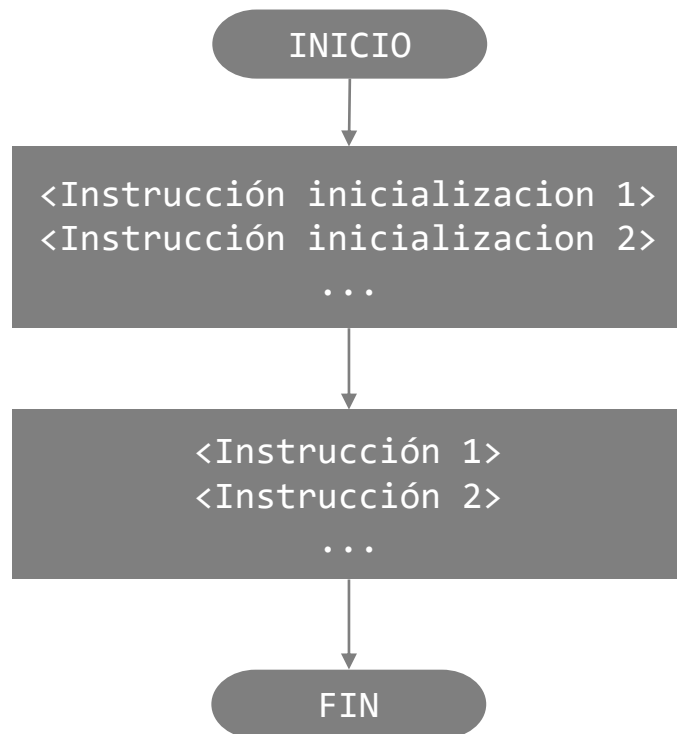
Definición de variables:

- **radio:** radio (Real)
- **area:** radio (Real)

¿Proceso?: Elevar el radio al cuadrado y multiplicarlo por π

Pseudocódigo (Opcional)

- Existe otra notación de pseudocódigo que sí declara el tipo de dato de cada variable (texto guía, notación del profesor Carlos Mera).
- La diferencia frente a la vista anteriormente: aquí la inicialización especifica `<Tipo_de_dato> <listaDeVariables>` en lugar de solo `<Instrucción inicialización>`.



Inicio

```
# Declaracion de variables  
<Tipo_de_dato_1> <listaDeVariables>  
<Tipo_de_dato_2> <listaDeVariables>  
...
```

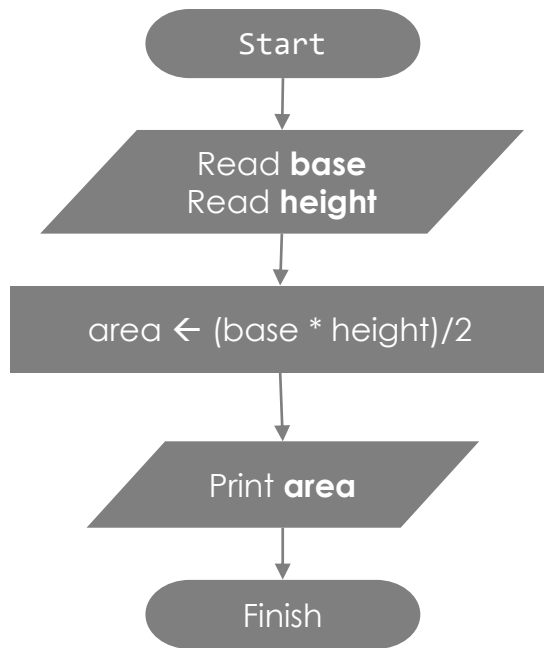
```
# Cuerpo del algoritmo  
<Instrucción 1>  
<Instrucción 2>  
...
```

Fin

Representación de algoritmos como diagramas de flujo

Ejemplo 1: Realice un programa que permita calcular al área de un triángulo

Diagrama de flujo



Pseudocódigo simple

Inicio

```
Leer(base)
Leer(height)
area = (base*height)/2
Escribir(area)
```

Fin

Pseudocódigo con declaración de variables

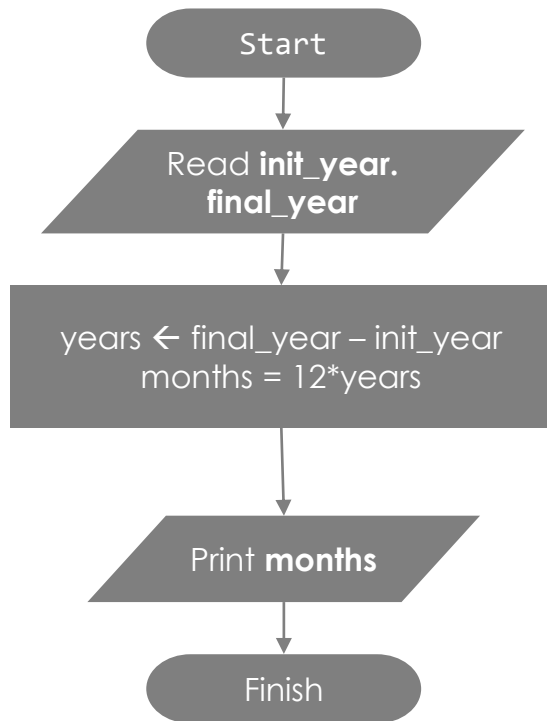
Inicio

```
Real base, height, area
Leer(base)
Leer(height)
area = (base*height)/2
Escribir(area)
```

Fin

Representación de algoritmos como diagramas de flujo

Ejemplo 2: Se requiere diseñar un algoritmo que **calcule el número de meses que hay entre los años A y B.**



Pseudocódigo simple

Inicio

```
Leer(init_year)
Leer(final_year)
years = final_year - init_year
months = 12*years
Escribir(months)
```

Fin

Pseudocódigo con declaración de variables

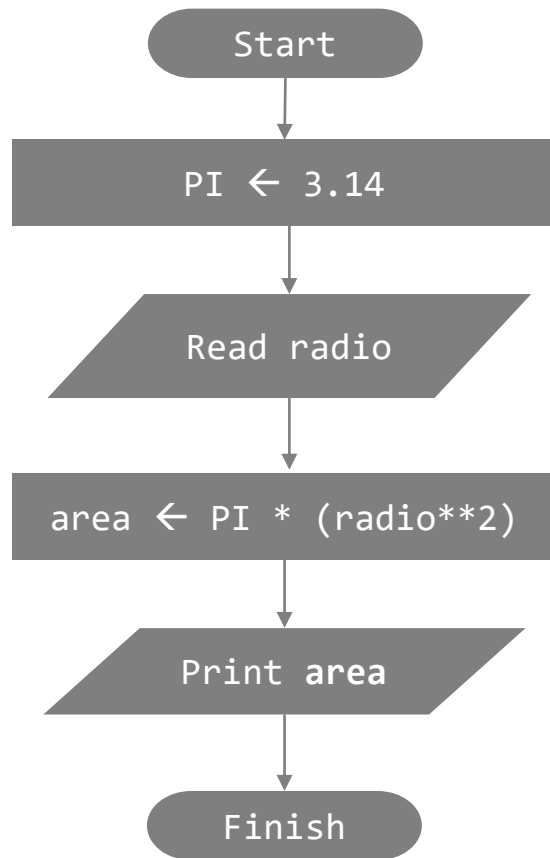
Inicio

```
Entero init_year, final_year
Entero years, months
Leer(init_year)
Leer(final_year)
years = final_year - init_year
months = 12*years
Escribir(months)
```

Fin

Representación de algoritmos como diagramas de flujo

Ejemplo 3: Hacer un algoritmo que calcule el área de un círculo.



Pseudocódigo simple

Inicio

```
PI = 3.14
Leer(radio)
area = PI*(radio**2)
Escribir(area)
```

Fin

Pseudocódigo con declaración de variables

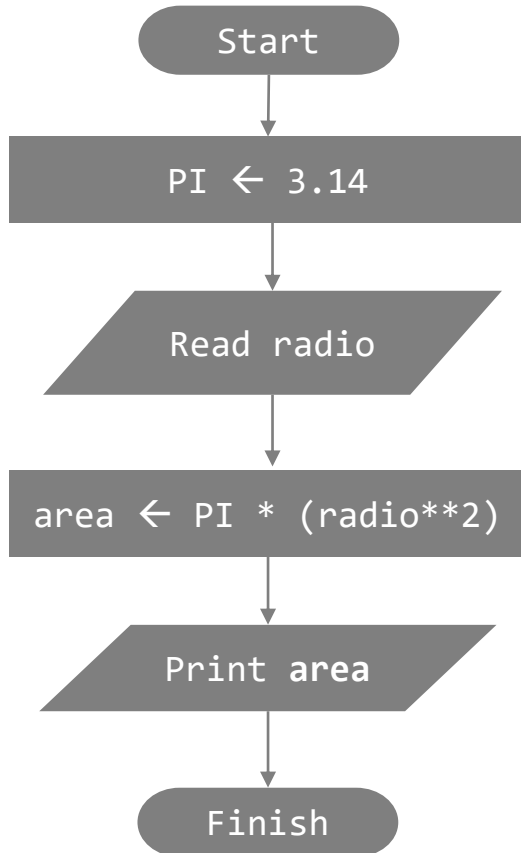
Inicio

```
Real PI = 3.14
Real radio, area
Leer(radio)
area = PI*(radio**2)
Escribir(area)
```

Fin

Sobre los programas

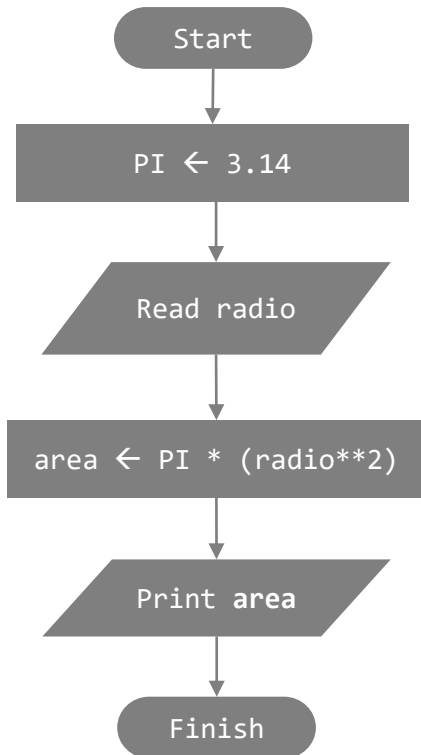
- Ya hemos logrado describir los pasos para plantear un algoritmo (algo natural para nosotros) de manera mas formal (diagrama de flujo y pseudocódigo), pero todavía no tenemos forma de ejecutarlo en una maquina.
- Necesitamos codificar el algoritmo en un lenguaje de programación para convertirlo en un programa ejecutable.



Programa en lenguaje Python

Problema:

Hacer un algoritmo que calcule el área de un círculo



Inicio

```
PI = 3.14  
Leer(radio)  
area = PI*(radio**2)  
Escribir(area)
```

Fin

Inicializacion

```
PI = 3.14
```

Entrada de datos

```
radio = float(input("Digite el radio: "))
```

Calculo del area

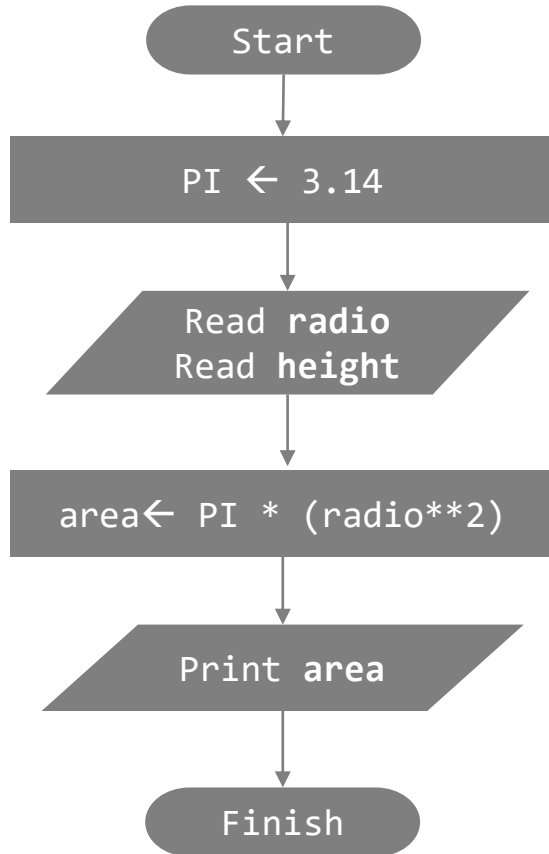
```
area = PI * (radio**2)
```

Despliegue del area

```
print("El área del círculo es:", area)
```

[link](#)

Problema - Solución



Inicio

```
Real PI = 3.14  
Real radio, area;  
Leer(radio)  
area = PI*(radio**2)  
Escribir(area)
```

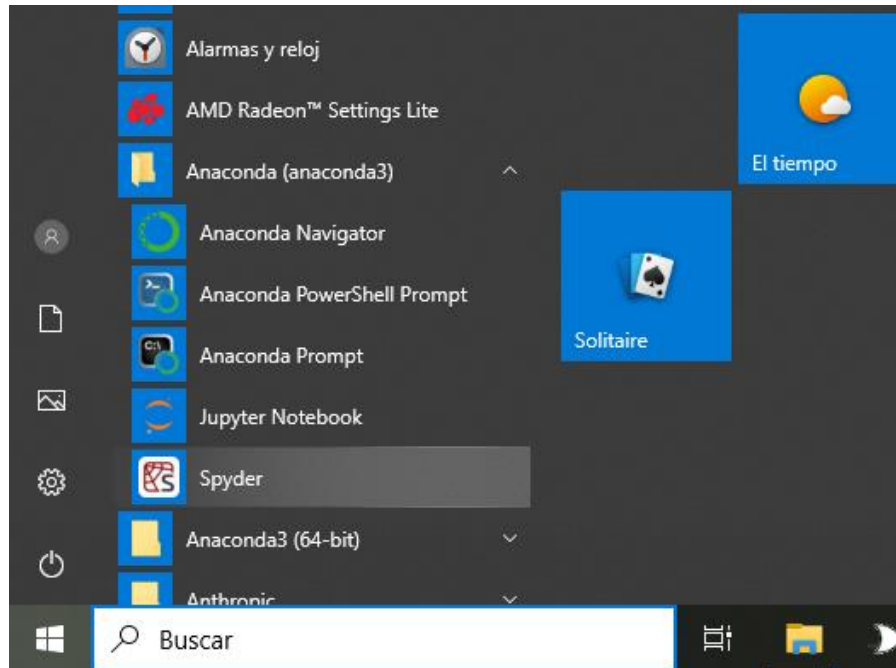
Fin

```
import java.util.Scanner;  
  
public class MainClass {  
    public static void main(String[] args) {  
        // Declaraciones  
        Scanner keyboard = new Scanner(System.in);  
        final double PI = 3.14;  
        double radio, area;  
        // Entrada de datos  
        System.out.print("Digite el radio: ");  
        radio = keyboard.nextDouble();  
        // Calculo del area  
        area = Math.PI * Math.pow(radio, 2);  
        // Despliegue del area  
        System.out.println("El área del círculo es: " + area);  
        keyboard.close();  
    }  
}
```


Codificando un programa en Python

Abrir Spyder

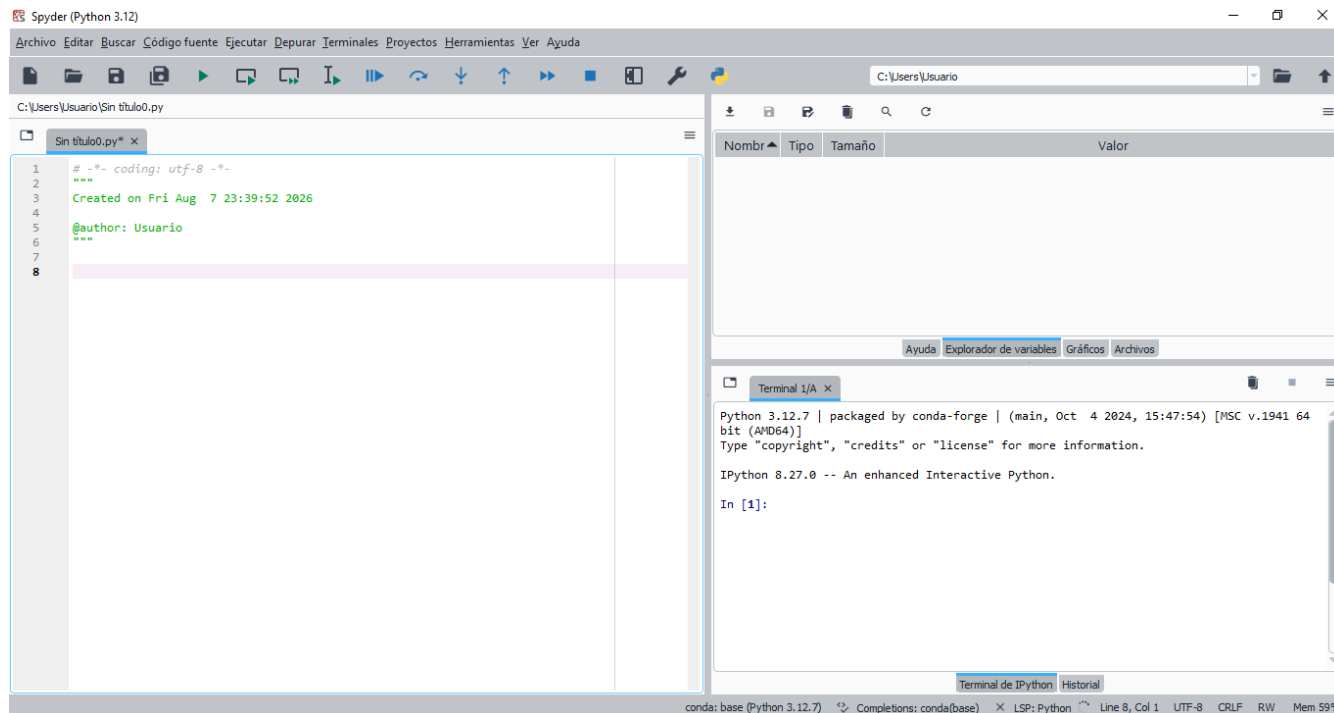
- **Spyder** se instala junto con **Anaconda** y se abre desde el menú de inicio (Anaconda3 → Spyder).
- Es el entorno de desarrollo (IDE) que se usará para escribir y ejecutar los programas de Python del curso.



Codificando un programa en Python

Crear un nuevo script

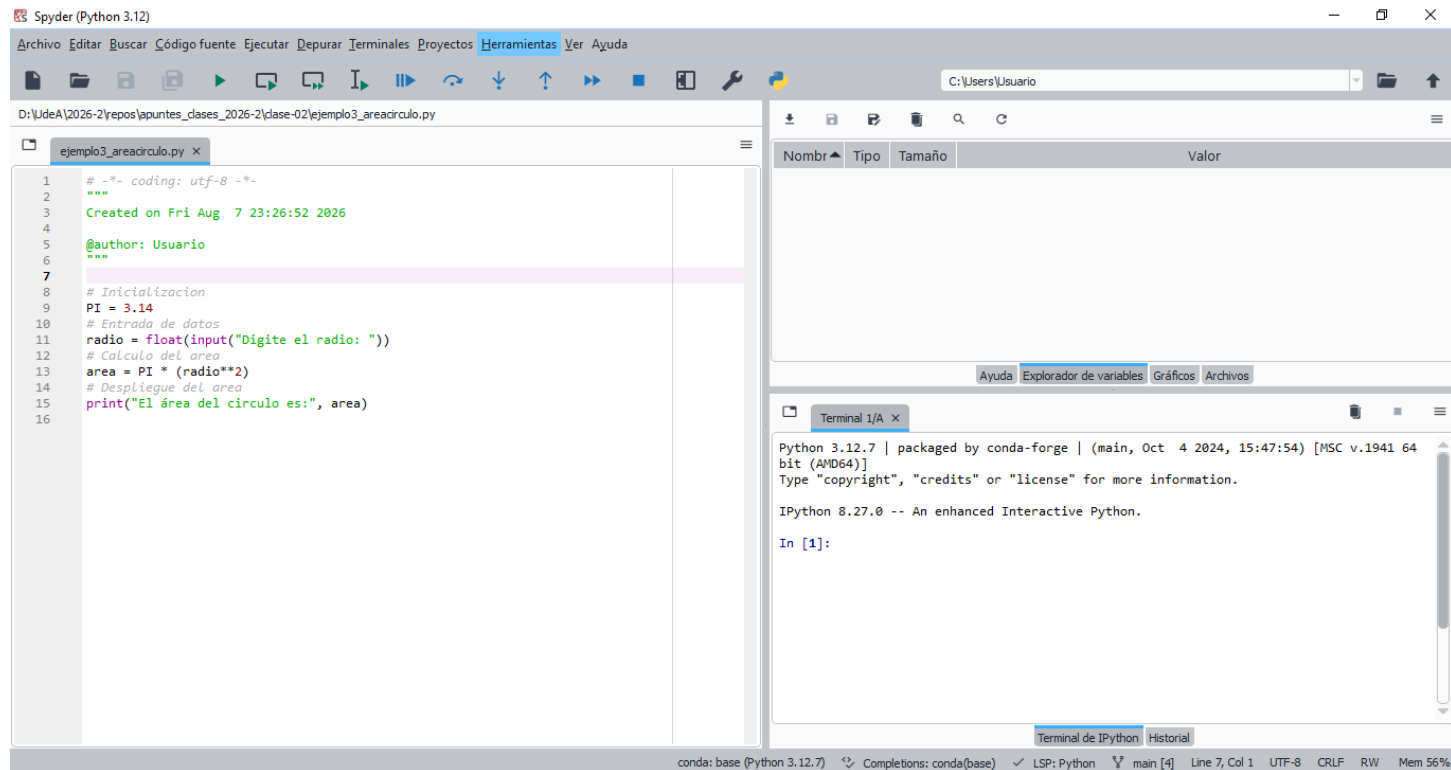
- Al abrir **Spyder** (o crear un archivo nuevo), se genera un script vacío con una plantilla inicial: codificación (utf-8), fecha de creación y autor.
- El nombre por defecto es `Sin título0.py` — se recomienda guardarlo con un nombre descriptivo (ej. `ejemplo3_areacirculo.py`) antes de escribir el código.
- En este punto, la consola y el explorador de variables están vacíos, porque aún no se ha ejecutado nada.



Codificando un programa en Python

Escribir el código

- El código se escribe en el editor (panel izquierdo), guardado como archivo .py (ej. ejemplo3_areacirculo.py).
- El botón  (**Ejecutar**) en la barra de herramientas corre el script completo.



Codificando un programa en Python

Ejecutar y ver resultados

- Al ejecutar, la consola solicita los datos de entrada definidos en el código (ej. "Digite el radio: ") y muestra la salida del programa.
- El explorador de variables se actualiza automáticamente, mostrando nombre, tipo, tamaño y valor de cada variable (PI, radio, area).
- Esto permite verificar visualmente que el programa calculó y almacenó los valores correctamente.

The screenshot displays the Spyder Python IDE interface. The main editor shows a Python script named `ejemplo3_areacirculo.py` with the following code:

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Fri Aug 7 23:26:52 2026
4
5  @author: Usuario
6  """
7
8  # Inicializacion
9  PI = 3.14
10 # Entrada de datos
11 radio = float(input("Digite el radio: "))
12 # Cálculo del área
13 area = PI * (radio**2)
14 # Despliegue del área
15 print("El área del círculo es:", area)
16
```

The Variable Explorer on the right shows the current state of the program's variables:

Nombr	Tipo	Tamaño	Valor
area	float	1	31400.0
PI	float	1	3.14
radio	float	1	100.0

The IPython terminal at the bottom shows the execution output:

```
IPython 8.27.0 -- An enhanced Interactive Python.
In [1]: runfile('D:/UdeA/2026-2/repos/apuntes_clases_2026-2/clase-02/
ejemplo3_areacirculo.py', wdir='D:/UdeA/2026-2/repos/apuntes_clases_2026-2/clase-02')
Reloaded modules: pydevd_plugins.extensions, pydevd_plugins.pydevd_line_validation,
pydevd_plugins.django_debug, pydevd_plugins.jinja2_debug,
pydevd_plugins.extensions.types, pydevd_plugins.extensions.types.pydevd_helpers,
pydevd_plugins.extensions.types.pydevd_plugin_numpy_types,
pydevd_plugins.extensions.types.pydevd_plugin_pandas_types,
pydevd_plugins.extensions.types.pydevd_plugins_django_form_str,
pydevd_plugins.extensions.pydevd_plugin_omegaconf
Digite el radio: 100
El área del círculo es: 31400.0
In [2]: |
```

The status bar at the bottom indicates the environment is `conda: base (Python 3.12.7)` and the file is `Line 7, Col 1`.

Ejercicios de repaso

1. Hacer un programa que calcule el área y el perímetro de un círculo dado su radio.
2. Hacer un programa que solicite el nombre, la identificación, el número de horas trabajadas y el valor por hora y realice el cálculo del salario neto teniendo en cuenta que se cobra un impuesto del 10% por salud y un 5% debido a pensiones.
3. Hacer un programa que pida el nombre y la edad (en años) y retorne un mensaje con el nombre y la edad en días.
4. Hacer un programa que solicite los 2 catetos de un triángulo rectángulo y calcule su hipotenusa.
5. Diseñar un algoritmo que lea dos valores reales y nos muestre los resultados de sumar, restar, dividir y multiplicar dichos números.
6. Si una persona corta sus uñas en promedio una vez por cada 1.5 semanas, dada la edad de una persona calcular cuantas veces ha cortado sus uñas en toda su vida. Adicionalmente, si en cada corte en promedio obtiene 0.5 gramos del material de la uña cuántos kilos lleva en su existencia.

W

3

18-20

✓

18-20

Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://inventwithpython.com/>

Eso es todo amigos

